

Lessons Learned

During the development of the Monster Trading Card Game (MTCG) REST server, several key lessons were learned:

1. Importance of Modular Design:

- Structuring the application with clear responsibilities for each class helped maintain readability and flexibility.
- The separation of services (e.g., user management, battle logic) streamlined development and testing.

2. Efficient Error Handling:

- Implementing robust error handling mechanisms (e.g., returning proper HTTP status codes) significantly improved the user experience and debugging process.

3. RESTful API Design:

- Designing API endpoints that adhere to REST principles ensured a predictable and consistent interface for clients.

4. Testing Integration Early:

- Introducing unit tests during the initial development phase identified edge cases early, reducing debugging efforts later.

Unit Testing Decisions

To ensure the reliability of the application, specific decisions were made for unit testing:

1. Testing Coverage:

- Unit tests were created for critical functionalities such as user registration, login, card management, and battle logic.
- Focused tests on edge cases, such as invalid user input or insufficient resources for trading.

2. Mocking Services:

- The use of mocking frameworks allowed isolation of individual components, ensuring tests focused on specific functionalities without dependencies on external services.

3. Automated Test Scripts:

- A comprehensive CURL-based test script validated end-to-end functionality, including REST API responses.

4. Continuous Feedback:

- Running tests during each development phase provided immediate feedback, ensuring new features didn't introduce regressions.
-

Unique Features

1. Dynamic Battle System:

- The game incorporates a battle system where players compete with randomly selected cards from their deck.
- A real-time calculation of damage and effectiveness ensures engaging gameplay.

2. Flexible Card Management:

- Users can configure their decks dynamically, promoting strategic gameplay.

3. Comprehensive Trading System:

- A trading feature allows users to create and fulfill trade offers, including filters for card type and damage requirements.

4. Seamless Integration with REST Standards:

- The server's adherence to RESTful principles ensures compatibility with a wide range of clients, including automated testing tools.